# Randomized Algorithms Notes

Wilson Pan

April 1, 2026

## Contents

**1 Jan 4** **2**
1.1 Background . . . 2
1.1.1 Permutations and Cycle Notation . . . 2
1.1.2 Probabilistic Tools . . . 2
1.2 100 Prisoners Problem: . . . 2
1.3 Cycle Following Algorithm . . . 3
1.3.1 A Warm-up Bound using Markov's Inequality . . . 3
1.3.2 Exact Probability Calculation . . . 3

**2 Jan 21** **6**
2.1 Mcdiarmid's Theorem . . . 6
2.2 Some theorem . . . 6

**3 Chernoff Bound (Feb 4)** **7**

**4 Feb 9** **8**

**5 Feb 11** **10**

**6 Feb 18** **12**

**7 Feb 23: Bourgain's Embedding Theorem** **14**

**8 March 10: Metric Embedding Day 2** **16**

**9 March 16** **18**

**10 March 18 — Optional Stopping Theorem** **20**

**11 March 23** **21**

**12 March 25** **22**
12.1 Linear probing hash table — setup . . . 22

**13 April 1: Succinct Data Structures** **23**

# 1 Jan 4

## 1.1 Background

Before analyzing the main problem, we briefly review two important concepts that will be central to our solution: the cycle decomposition of permutations and Markov's Inequality.

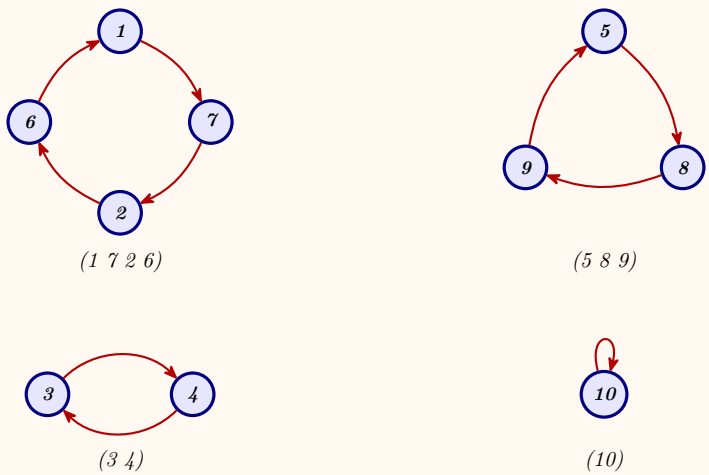
### 1.1.1 Permutations and Cycle Notation

A permutation $\sigma$ of the set $\{1, \ldots, n\}$ is a bijection from the set to itself. We can decompose any permutation into disjoint cycles.

**Example 1.1.** *Consider the permutation defined by:*

$$\sigma = \begin{pmatrix} 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 & 10 \\ 7 & 6 & 4 & 3 & 8 & 1 & 2 & 9 & 5 & 10 \end{pmatrix}$$

*In disjoint cycle notation, this is written as* $(1\ 7\ 2\ 6)(3\ 4)(5\ 8\ 9)(10)$. *We can visualize this decomposition below:*




### 1.1.2 Probabilistic Tools

We will also make use of Markov's Inequality to bound probabilities.

**Theorem 1.2** (Markov's Inequality)**.** *If $X$ is a non-negative random variable and $k > 0$ then:*

$$\mathbb{P}[X \geq k \cdot \mathbb{E}[X]] \leq \frac{1}{k}.$$

## 1.2 100 Prisoners Problem:

In a prison, there are 100 prisoners and there is a warden who want to retire. He decides to play a game.

1. In a room there are 100 boxes, In these boxes, is a permutation of the numbers $1, 2, ..., 100$. The boxes are closed and the prisoners cannot see the numbers within the boxes.
2. The prisoners will each enter and play a mini-game. For the $i$-th prisoner, they open 50 boxes and wins by finding the number $i$.
3. After a prisoner leaves, the boxes are closed again, and the state of the room is restored to exactly how it was before they entered. Prisoners cannot communicate after the game begins.
4. Prisoners win the game (and survive) if and only if **every** prisoner finds their own number. Otherwise, they are all executed.

**Naive Algorithm:** Each prisoner opens 50 random boxes then

$$\mathbb{P}[\text{Victory}] = \left(\frac{1}{2}\right)^{100} \approx 8 \times 10^{-31}.$$

## 1.3 Cycle Following Algorithm

We propose a strategy that utilizes the permutation structure inside the boxes.
**Cycle Algorithm:** Consider the algorithm where person $i$ opens box $i$, sees $j$, then open box $j$, see $k$ and so on. This continues until they find their number or exhaust their 50 tries.

**Lemma 1.3.** *Under the Cycle Algorithm, prisoner $i$ succeeds if and only if $i$ belongs to a cycle of length less than or equal to 50. Consequently, the team achieves victory if and only if the permutation contains no cycle of length strictly greater than 50.*

### 1.3.1 A Warm-up Bound using Markov's Inequality

Before calculating the exact probability, let us try to bound the failure probability of a slightly easier version of the game using Markov's Inequality.

**Lemma 1.4.** *Consider a variant where each prisoner is allowed to open 75 boxes. Using the Cycle Algorithm, the probability of failure is at most* $1/3$.

*Proof.* Let $X$ be the number of prisoners who lose. We write

$$X = \sum_{i=1}^{100} I_i \text{ where } I_i = \begin{cases} 1 & \text{if prisoner } i \text{ loses} \\ 0 & \text{otherwise} \end{cases}.$$

First, observe that for any prisoner $i$, the probability that their specific number lies in the 25 unopened boxes is exactly $25/100 = 1/4$. Thus, $\mathbb{E}[I_i] = 1/4$. By linearity of expectation:

$$\mathbb{E}[X] = \sum_{i=1}^{100} \mathbb{E}[I_i] = 100 \cdot \frac{1}{4} = 25.$$

Now, observe the Cycle Algorithm: If prisoner $i$ loses, they must be part of a cycle of length greater than 75. As cycles are disjoint, anyone else with a number in that same cycle will also lose. Therefore, if anyone loses like when $X > 0$, then strictly more than 75 people must lose so $X > 75$.
Using Markov's Inequality with $k = 3$:

$$\mathbb{P}[\text{Non-victory}] = \mathbb{P}[X > 0] = \mathbb{P}[X \geq 76] \leq \mathbb{P}[X \geq 75] \leq \frac{\mathbb{E}[X]}{75} = \frac{25}{75} = \frac{1}{3}.$$

□

### 1.3.2 Exact Probability Calculation

**Theorem 1.5.** *The Cycle Algorithm achieves a victory probability of* $\mathbb{P}[\textit{Victory}] \approx 0.31$.

*Proof.* The prisoners fail if and only if there exists a cycle of length $\ell > 50$. Since the sum of cycle lengths is 100, there can be at most one cycle of length greater than 50. Thus, the events of having a cycle of length $\ell$ (for $\ell > 50$) are mutually exclusive.

$$\mathbb{P}[\text{Non-victory}] = \mathbb{P}[\exists \text{ cycle of length } > 50] = \sum_{\ell=51}^{100} \mathbb{P}[\exists \text{ cycle of length } \ell].$$

To calculate the number of permutations of $N = 100$ elements that contain a cycle of length exactly $\ell$.

1. Choose $\ell$ elements to be in the cycle: $\binom{N}{\ell}$ ways

2. Arrange these $\ell$ elements into a cycle: $(\ell - 1)!$ ways
3. Permute the remaining $N - \ell$ elements arbitrarily: $(N - \ell)!$ ways

The total number of such permutations is:

$$\binom{N}{\ell} \cdot (\ell - 1)! \cdot (N - \ell)! = \frac{N!}{\ell!(N - \ell)!}(\ell - 1)!(N - \ell)! = \frac{N!}{\ell}.$$

The probability that a random permutation contains a cycle of length $\ell$ is therefore:

$$\frac{N!/\ell}{N!} = \frac{1}{\ell}.$$

Summing these probabilities:

$$\mathbb{P}[\text{Non-victory}] = \sum_{\ell=51}^{100} \frac{1}{\ell} = H_{100} - H_{50} \approx \ln(100) - \ln(50) = \ln(2) \approx 0.69.$$

Thus, the probability of victory is $1 - 0.69 \approx 0.31$. □

**Theorem 1.6.** *The previous algorithm is optimal.*

*Proof.* Consider a second version of the game:

1. Prisoner 1 enters, opens 50 boxes
2. The difference between the games is that the boxes **remain open** for subsequent prisoners
3. Prisoner $i$ enters. They win immediately if their box was already opened. Otherwise they may open more boxes until 50 total boxes (including those opened by previous prisoners) have been opened in the room

We rely on the following two lemmas to complete the theorem. □

**Lemma 1.7.** *The Cycle Algorithm achieves victory in Game 2 if and only if there are no cycles of length greater than 50.*

*Proof.* Consider when there are no cycles of length $> 50$, the Cycle Algorithm wins Game 1, and since Game 2 provides more information as boxes stay opened, it also wins Game 2. Conversely, suppose there is a cycle of length $> 50$. The first prisoner in that cycle to enter the room will be forced to open the boxes in that cycle, since the boxes were closed before they entered or if open, did not contain their number, they must follow the cycle. They will run out of their 50 openings before finding their number. Thus, the team fails. Therefore, the performance of the Cycle Algorithm is identical in Game 1 and Game 2. □

**Lemma 1.8.** *In Game 2, all strategies yield the same probability of victory.*

*Proof.* In Game 2, the boxes remain open, this means the decision of which box to open next depends on the set of boxes currently revealed. However, since the permutation inside the boxes is chosen uniformly, the values hidden in the unopened boxes are a random permutation of the remaining numbers.
Consequently, the probability that the *next* box opened contains a specific required number depends only on the number of boxes remaining, not on which specific box is chosen. Regardless of the strategy used, the probability that the union of opened boxes contains the set of all prisoners' numbers is not changed. □

**Conclusion of Optimality:** We have established the following chain of inequalities:

$$\mathbb{P}(\text{Cycle wins Game 1}) \leq \mathbb{P}(\text{Optimal Strategy wins Game 1}) \leq \mathbb{P}(\text{Optimal Strategy wins Game 2}).$$

The second inequality is true as any strategy for Game 1 is a strategy for Game 2 if we ignore the extra information, so Game 2 is strictly easier. However, by Lemma 4.3, all strategies in Game 2 are equivalent.

$$\mathbb{P}(\text{Optimal Strategy wins Game 2}) = \mathbb{P}(\text{Cycle wins Game 2}).$$

And by Lemma 4.2:

$$\mathbb{P}(\text{Cycle wins Game 2}) = \mathbb{P}(\text{Cycle wins Game 1}).$$

Combining these we get:

$$\mathbb{P}(\text{Cycle wins Game 1}) \leq \mathbb{P}(\text{Optimal Strategy wins Game 1}) \leq \mathbb{P}(\text{Cycle wins Game 1}).$$

Therefore, the inequalities must be equalities, and the Cycle Algorithm is optimal for Game 1.

# 2 Jan 21

## 2.1 Mcdiarmid's Theorem

**Theorem 2.1.** *Mcdiarmid's Theorem (1993) Given 3-colorable graph, we can efficiently compute a triangle-free partition.*
*The algorithm is in each round*
1. *Find some triangle in some part*
2. *Pick random vertex in the triangle*
3. *Move vertex to other side*

*Proof.* Pick a specific 3 coloring of the graph then let $X = \#$ color 1 vertices on left and $Y = \#$ color 2 vertices on right side. Observe that the value $X + Y$ changes by $\{-1, 0, 1\}$ with equal probability. Then observe a random walk in $[0, n]$

**Lemma 2.2.** *Let $T(n) = \mathbb{E}[\text{time to get to } \pm n]$ then $T(n) = n^2$, but we restrict it to powers of 2. We can easily see the following recurrence holds and it implies the lemma*

$$T(n) = T\left(\frac{n}{2}\right) + T\left(\frac{n}{2}\right) + \frac{1}{2}T(n).$$

So the expected number of rounds is $O\left(n^2\right)$ □

## 2.2 Some theorem

**Theorem 2.3.** *Given $G = (V, E)$ we want for each pair $x, y \in V$ to calculate $d(x, y) \pm 2$. We will do it in $O(n^{2.5} \ln n)$.*

*Proof.* Consider the shortest path $x \to y$

Option 1: Path has no vertices with degree $> \sqrt{n}$. We can run BFS from each node, but exclude high degree nodes from search. So $O(|V| \cdot n^{1.5}) = O(n^{2.5})$

Option 2: There exist a high degree vertex on $x \to y$ path. Define $S = 100\sqrt{n}\log n$ vertices, each of which is an independent uniform random vertex.

Claim: With good probability, every high-degree vertex is incident to at least one vertex in $S$.
Observe that there exist $s \in S$ such that

$$d(x, s) + d(s, y) \leq d(x, y) + 2.$$

For each $s \in S$ do BFS and this runs at $O(n^{2.5} \log n)$ Then for each $x, y$ compute

$$\min_{s \in S} d(x, s) + d(s, y).$$

This also is $O(n^{2.5} \log n)$.

$$\begin{aligned}
\Pr[\text{given high degree vertex } w \not\sim S] &\leq \Pr[\text{every sample misses neigh}(c_v)] \\
&\leq \left(1 - \frac{1}{\sqrt{n}}\right)^{100\sqrt{n}\log n} \\
&\approx \left(\frac{1}{e}\right)^{100\log n} \leq \frac{1}{n^{100}}
\end{aligned}$$

□

# 3 Chernoff Bound (Feb 4)

**Theorem 3.1.** *Let $X_1, ..., X_n$ be i.i.d uniformly randomly in $\{0, 1\}$, let $X = \sum_{i=1}^{n} X_i$ then*

$$\Pr\left[X \geq \frac{n}{2} + k\sqrt{n}\right] \leq e^{-\frac{k^2}{2}}.$$

**Theorem 3.2.** *Let $X_1, ..., x_n$ be i.i.d random variables $\{-1, 1\}$ and $X = \sum_{i=1}^{n} X_i$ then*

$$\Pr\left[X \geq k\sqrt{n}\right] \leq e^{-\frac{k^2}{2}}.$$

**Theorem 3.3.** *Quick Sort:*

$$\Pr[depth \geq 100 \log n] \leq \sum_{elts\ v} \Pr[elt\ v has\ recursion\ depth\ 100 \log n] \leq \frac{1}{n^2}.$$

*Let $P_1, ...,$ be subproblems containing $v$ at levels $1, 2, ....$ If pivot in middle half ($\Pr \geq \frac{1}{2}$) then $|P_{i+1}| \leq \frac{3}{4}|P_i|$.*
*Define*

$$X_i = \begin{cases} 1 & if\ P_i\ exist\ and\ |P_i| > \frac{3}{4}|P_{i-1}| \\ 0 & otherwise \end{cases}.$$

*We have*

$$\Pr\left[X_i = 1 \mid X_1, ..., X_{i-1}\right] \leq \frac{1}{2}.$$

*So*

$$\Pr\left[\sum_{i=1}^{100 \log n} X_i \geq 750 \log n\right] \leq e^{-1000 \log n/32} = e^{-30 \log n}.$$

# 4 Feb 9

**Lemma 4.1.** *(Poor Man's Chernoff Bound)*

$$\Pr[X \geq 2k\sqrt{n}] \leq \frac{1}{2^k}.$$

*Proof.* Note:

$$\Pr[X \geq 2\sqrt{n}] \leq \frac{1}{4} \implies \Pr[\text{any prefix of } X \text{ is } \geq 2\sqrt{n}] \leq \frac{1}{2}. \qquad \text{(Chebyshev's Inequality)}$$

Let $t_i$ be the event that the prefix of $X$ reaches $2i\sqrt{n}$ then

$$\Pr[t_i \text{ exists} | t_{i-1} \text{ exist}] \leq \frac{1}{2}.$$

So

$$\Pr[X \geq 2k\sqrt{n}] \leq \Pr[t_k \text{ exists}] \leq \frac{1}{2^k}.$$

□

**Lemma 4.2.** *Let $X_1, ..., X_n$ be i.i.d random variables where each $X_i$ satisfies*

$$\Pr[X_i \geq j] \leq p^j.$$

*Then $X = \sum_{i=1}^n X_i$ satisfies*

$$\Pr[X \geq 2n] \leq (4p)^n.$$

*Proof.*

$$\sum X_i \geq 2n \implies \sum \lfloor X_i \rfloor \geq n \implies \exists \vec{Y} = (Y_1, ..., Y_n) \text{ such that } X_i \geq Y_i \text{ for all } i \text{ and } \sum Y_i = n.$$

FOr a given $\vec{Y} = (Y_1, ..., Y_n)$ we have

$$\Pr[\vec{Y} \text{ occurs}] = \Pr[X_i \geq Y_i \ \forall i] = \prod_i \Pr[X_i \geq Y_i] \leq \prod_i p^{Y_i} = p^n.$$

We can encode $\vec{Y}$ as a binary string by stars and bars of length $2n$.

$$\binom{2n}{n} \leq 2^{2n}.$$

So number of such $\vec{Y}$ is at most $4^n$. So

$$\Pr[X \geq 2n] \leq \sum_{\vec{Y}} \Pr[\vec{Y} \text{ occurs}] \leq 4^n \cdot p^n = (4p)^n.$$

□

**Theorem 4.3.** *The (Anti-Chernoff Bound)*

$$\Pr[X \geq \frac{k}{4}\sqrt{n}] \geq \frac{1}{4^{k^2}}.$$

*Proof.* We'll prove using the core fact

$$\Pr[X \geq \frac{1}{4}\sqrt{n}] \geq \frac{1}{4}.$$

We can split the coin flips into $k^2$ groups of $m = \frac{n}{k^2}$. Let $C_i = \sum(\text{coins in group } i)$ then

$$\Pr[C_i \geq \frac{1}{4}\sqrt{m}] \geq \frac{1}{4} \implies \Pr[\text{every } C_i \text{ is } \geq \frac{1}{4}\sqrt{m}] \geq \frac{1}{4^{k^2}}.$$

If this occurs $X = \sum C_i \geq k^2(\frac{1}{4}\sqrt{m}) = \frac{k}{4}\sqrt{n}$. □

**Theorem 4.4.**

$$\Pr[X \geq 16k\sqrt{n}] \leq \frac{1}{2^{k^2}}.$$

*Proof.* Define $C_1, C_2, \ldots$ as before. By poor man's chernoff bound: $\Pr[C_i \geq 2j\sqrt{m}] \leq \frac{1}{2^j}$. Define $Y_i = \max(0, \frac{C_i}{8\sqrt{m}})$.

$$\Pr[C_i \geq 2j\sqrt{m}] = \Pr\left[Y_i \geq \frac{j}{4}\right].$$

So

$$\Pr[Y_i \geq j] \leq \frac{1}{2^{4j}} = \frac{1}{16^j}.$$

By lemma 4.2:

$$\Pr\left[\sum Y_i \geq 2k^2\right] \leq \left(4 \cdot \frac{1}{16}\right)^{k^2}.$$

□

# 5 Feb 11

**Theorem 5.1.** *Let $p \in [0, \frac{1}{2}]$ and let $X_1, ..., X_n \in \{0, 1\}$ be i.i.d coin flips such that $\Pr[X_i = 1] = p$ for all $i$. Let $X = \sum_{i=1}^n X_i$.*
*Small Deviation Regime: For $k \in [1, \sqrt{\mu}]$*

$$\Pr[X \geq \mu + k\sqrt{\mu}] \leq e^{-\theta(k^2)} \leq 2^{-\frac{k^2}{2}}.$$

*Large Deviation Regime: For $J \geq 2$ such that $1 \leq J\mu \leq n$*

$$\Pr[X \geq J\mu] = \left(\frac{1}{J}\right)^{\theta(J\mu)} \leq \left(\frac{1}{J}\right)^{J\mu}.$$

**Theorem 5.2.** *Adapted Version of Chernoff Bound: Same upper bound holds if $X_1, ..., X_n$ are independent $[0, 1]$ random variables such that $\mathbb{E}[\sum X_i] = \mu$ or the independent condition can be dropped as long as*

$$\Pr[X_i = 1 | X_1, ..., X_{i=1}] \leq p.$$

**Theorem 5.3.** *Anti Chernoff Bound:*

$$\Pr[X \geq J\mu] \geq \Omega\left(\frac{1}{J}\right)^{J\mu}.$$

*We can break coins into $J\mu$ chunks*

**Lemma 5.4.** *Let $\ell \in \mathbb{N}$ then*

$$\Pr[C_i \geq \ell] \leq \frac{1}{J^\ell}.$$

*Proof.*

$$\Pr[C_i \geq 1] \leq \mathbb{E}[C_i] \leq \frac{1}{J}.$$

Let $t_i$ be the number of flips it takes to have a total sum $i$. Then

$$\Pr[t_i \text{ exists}] \leq \frac{1}{J} \text{ and } \Pr[t_i \text{ exists} | t_{i-1} \text{ exists}] \leq \frac{1}{J}.$$

□

**Theorem 5.5.** *Azuma's Inequality: Let $Y_1, ..., Y_n \in [-1, 1]$ be random variables such that $\mathbb{E}[Y_i | Y_1, ..., Y_{i-1}] = 0$ then we have $\Pr[\sum Y_i \geq k\sqrt{n}] \leq e^{-\frac{k^2}{2}}$*

**Theorem 5.6.** *McDiarmid's Inequality: Let $X_1, ..., X_n$ be independent random variables. Suppose $f$ is 1-lipchitz for any $X_1, ..., X_n$ and $\overline{X_i}$ then*

$$|f(X_1, ..., X_i, ..., X_n) - f(X_1, ..., \overline{X_i}, ..., X_n)| \leq 1.$$

*Then*

$$\Pr[f - \mathbb{E}[f] \geq k\sqrt{n}] \leq e^{-\frac{k^2}{2}}.$$

*Proof.* Define $Y_1 = \mathbb{E}[f|X_1] - \mathbb{E}[f]$, $Y_2 = \mathbb{E}[f|X_1, X_2] - \mathbb{E}[f|X_1]$ and in general $Y_i = \mathbb{E}[f|X_1, ..., X_i] - \mathbb{E}[f|X_1, ..., X_{i-1}]$.
Then

$$\sum_{i=1}^n Y_i = \mathbb{E}[f|X_1, ..., X_n] - \mathbb{E}[f].$$

We have each $Y_i \in [-1, 1]$ and we need to prove $\mathbb{E}[Y_i | X_1, ..., X_{i-1}] = 0$.
Fix $X_1, ..., X_{i-1}$. Let $X_i$ still be random then by law of total expectation

$$\underbrace{\mathbb{E}[\overbrace{\mathbb{E}[f|X_1, ..., X_i]}^{P}]}_{\text{random over } X_i} = \underbrace{\mathbb{E}[f|X_1, ..., X_{i-1}]}_{Q}.$$

So

$$\mathbb{E}[Y_i] = \mathbb{E}[P - Q|X_1, ..., X_{i-1}] = 0.$$

$\square$

# 6 Feb 18

**Proposition 6.1.** *The Cup Game:*
1. *There are $n$ cups that are initially empty*
2. *In each step, the filler distributes up to $1-\epsilon$ units of water for $\epsilon \in [0, \frac{1}{2}]$*
3. *The emptier removes up to 1 unit from single cup.*
4. *The emptier's goal is to minimize backlog (amount of water in the fullest cup)*
5. *Filler is a oblivious adversary that does not adapt to behavior of emptier and precommits to what it will do on each time step*

**Remark 6.2.** *The greedy algorithm for the emptier is to empty up to 1 unit from fullest cup. This was proven in 1987 by Dietz/Slaeator where Greedy achieves backlog of $O(\log n)$ and the filler can force backlog $\Omega(\log n)$ against any deterministic algorithm. The filler achieves this by*
1. *Put $\frac{1-\epsilon}{n}$ water in each cup*
2. *Emptier empties from some cup*
3. *Filler puts $\frac{1-\epsilon}{n-1}$ water in each remaining cup*

*Continuing this the backlog after $n-1$ steps is*

$$(1-\epsilon)\left(\frac{1}{n}+\frac{1}{n-1}+\cdots+\frac{1}{2}\right)=\theta(\log n).$$

**Theorem 6.3.** *(Bender, Foreach-Colton, Kuszmaul) Let $\epsilon \in \theta(1)$. There is a random algorithm that at any given moment, guarantees backlog $O(\log\log n)$ with high probability.*

*Proof.* Smoothed Greedy Algorithm for Emptier:

1. At the start, pick random i.i.d $r_1, ..., r_n \in [0,1]$ and put $r_i$ water in cup $i$.
2. In each step, empty 1 unit from fullest cup but if every cup has $< 1$ unit then skip our turn.

Let $a_i(t)$ be the amount of water the filler puts in cup $i$ on step $t$, $b_i(t)$ be the amount of water the emptier removes from cup $i$ on step $t$ and $c_i(t)$ be the fill of cup $i$ after step $t$.

**Claim 6.4.** *For all $t, i, c_i(t) \pmod 1$ is uniformly random.*

*Proof.*

$$c_i(t) = \underbrace{r_i}_{\text{random}} + \underbrace{\sum_{\ell=1}^{t} a_i(\ell)}_{\text{fixed}} - \underbrace{\sum_{\ell=1}^{t} b_i(\ell)}_{\text{integer}}.$$

□

Define a potential function $\phi(t) = \sum_{i=1}^{n} \lfloor c_i(t)^2 \rfloor$

**Lemma 6.5.** *Consider time $t_1$, let $t_0 < t_1$ be most recent time that $\phi(t_0)$ was zero. With high probability, $t_1 - t_0 = O(\log n)$.*

*Proof.* For the steps $t_0+1, ..., t_1-1$, the emptier decreases $\phi$ by exactly 1 in each step. So the emptier decreases $\phi$ by $t_1 - t_0 - 1$.
For the filler

$$\Pr[\text{cup } i \text{ crosses an int threshold}] = a_i(t).$$

# of integer thresholds crossed is a sum of independent $[0,1]$ random variables with total mean $\sum a_i(t) \leq (1-\epsilon)$.
Let $X_i$ be the amount of water that gets added to each cup during the $k$ steps. Then the total number of thresholds that the cup $i$ crosses is $\lfloor X_i \rfloor$ for sure and one more with probability $X_i - \lfloor X_i \rfloor$.
Total incremenet across all cups is the sum of independent $[0,1]$ random variables with mean $X =$

$\sum X_i \leq (1-\epsilon)k$.
By Chernoff Bound,

$$\Pr[X \geq k-1] \leq e^{-\Omega(k)}.$$

During steps $t_0+1, ..., t_1-1$, suppose $t_1 - t_0 = k$ then emptier decreases by $t_1 - t_0 - 1 = k-1$. So the filler must have increased it by $\geq k-1$

$$\Pr[X] \leq e^{-\Omega(k)} \implies \Pr[t_1 - t_0 = k] \leq e^{-\Omega(k)} \implies t_1 - t_0 = O(\log n).$$

□

Consider steps $t_0+1, ..., t_1-1$, all cups are initially $< 1$ and consider only cups that go above 2. The number of such cups is $\leq O(\log n)$ as $t_1 - t_0 = O(\log n)$ with high probability.
On these $O(\log n)$ cups if we consider just water above 2, using greedy algorithm implies that the backlog is $O(\log \#\text{cups involved}) = O(\log \log n)$. □

# 7 Feb 23: Bourgain's Embedding Theorem

**Definition 7.1.** *Given $(X_1, d_1)$ and $(X_2, d_2)$, a map $\phi : X_1 \to X_2$ has distortion $\alpha \geq 1$ if there exists a scaling parameter $\beta > 0$ such that for all $x, y \in X_1$:*

$$\beta d_1(x, y) \leq d_2(\phi(x), \phi(y)) \leq \alpha\beta d_1(x, y).$$

**Theorem 7.2** (Bourgain's Theorem)**.** *Let $(X, d)$ be any $n$-point metric space. Then there exists an embedding $\phi : (X, d) \to (\mathbb{R}^{O(\log^2 n)}, d_{\ell_1})$ such that the distortion is $O(\log n)$.*

**Definition 7.3.** *For $i = 1, \ldots, \log n$ and $j = 1, \ldots, c \log n$ (for a sufficiently large constant $c$), define a random subset $S_{i,j} \subseteq X$ by including each $x \in X$ independently with probability $\frac{1}{2^i}$.*
*We define the embedding $\phi : X \to \mathbb{R}^{\log n \times c \log n}$ as:*

$$\phi(x) = \big(d(x, S_{1,1}), d(x, S_{1,2}), \ldots, d(x, S_{\log n, c \log n})\big).$$

*where $d(x, S) = \min_{s \in S} d(x, s)$.*

**Theorem 7.4.** *With high probability in $n$, for all $x, y \in X$:*

$$\Omega(\log n) d(x, y) \leq d_{\ell_1}(\phi(x), \phi(y)) \leq c \log^2 n \, d(x, y).$$

*This yields an embedding with scaling factor $\beta = \Omega(\log n)$ and distortion $\alpha = O(\log n)$.*

**Remark 7.5.** *Recall that the $\ell_1$ distance is the sum of absolute differences across all coordinates. Thus, we need to analyze:*

$$d_{\ell_1}(\phi(x), \phi(y)) = \sum_{i=1}^{\log n} \sum_{j=1}^{c \log n} |d(x, S_{i,j}) - d(y, S_{i,j})|.$$

*The upper bound is immediate from the triangle inequality: $|d(x, S) - d(y, S)| \leq d(x, y)$. Summing this over all $c \log^2 n$ coordinates gives the upper bound $c \log^2 n \, d(x, y)$. The core of the proof is establishing the lower bound.*

*Proof.* Fix $x, y \in X$. For any $z \in X$ and $r \geq 0$, let $B(z, r) = \{w \in X \mid d(z, w) \leq r\}$ be the closed ball, and let $B^\circ(z, r) = \{w \in X \mid d(z, w) < r\}$ be the open ball.
For each $i$, define $\rho_i(x) = \inf\{r \mid |B(x, r)| \geq 2^i\}$. By definition, the open ball is strictly smaller than $2^i$, so $|B^\circ(x, \rho_i(x))| < 2^i$, while the closed ball satisfies $|B(x, \rho_i(x))| \geq 2^i$.
Define $r_i = \max(\rho_i(x), \rho_i(y))$. Without loss of generality, assume $r_i = \rho_i(x)$. This gives us two critical observations:

1. $|B^\circ(x, r_i)| < 2^i$ (since $r_i = \rho_i(x)$).
2. $|B(y, r_i)| \geq 2^i$ (since $r_i \geq \rho_i(y)$). Since balls only grow with radius, it also holds that $|B(y, r_{i-1})| \geq 2^{i-1}$.

Define $t$ as the largest integer such that $r_t \leq \frac{d(x,y)}{3}$. Define $r_0 = 0$, and cap the sequence by defining $r_{t+1} = \frac{d(x,y)}{3}$.

**Lemma 7.6.** *For any $i \in \{1, \ldots, t+1\}$ and $j \in \{1, \ldots, c \log n\}$, with constant probability $p > 0$ (specifically $p > \frac{1}{10}$),*

$$|d(x, S_{i,j}) - d(y, S_{i,j})| \geq r_i - r_{i-1}. \qquad (*)$$

*Proof of Lemma:* To satisfy $(*)$, it is sufficient for $S_{i,j}$ to miss the open ball $B^\circ(x, r_i)$ and hit the closed ball $B(y, r_{i-1})$. Since $r_i \leq \frac{d(x,y)}{3}$ and $r_{i-1} \leq \frac{d(x,y)}{3}$, the distance between the centers of these two balls is $d(x, y) \geq r_i + r_{i-1} + \frac{1}{3} d(x, y)$. Thus, $B^\circ(x, r_i)$ and $B(y, r_{i-1})$ are completely disjoint, making hitting and missing them completely independent events.

- **Miss Event:** $\Pr[S_{i,j} \cap B^\circ(x, r_i) = \emptyset] = \left(1 - \frac{1}{2^i}\right)^{|B^\circ(x,r_i)|} \geq \left(1 - \frac{1}{2^i}\right)^{2^i} \approx \frac{1}{e}$.
- **Hit Event:** $\Pr[S_{i,j} \cap B(y, r_{i-1}) \neq \emptyset] = 1 - \left(1 - \frac{1}{2^i}\right)^{|B(y,r_{i-1})|} \geq 1 - \left(1 - \frac{1}{2^i}\right)^{2^{i-1}} \approx 1 - \frac{1}{\sqrt{e}}$.

By independence, the probability that both occur is at least $\frac{1}{e}\left(1 - \frac{1}{\sqrt{e}}\right) \approx 0.14 > \frac{1}{10}$. When this happens, $d(x, S_{i,j}) \geq r_i$ and $d(y, S_{i,j}) \leq r_{i-1}$, so the absolute difference is at least $r_i - r_{i-1}$.

**Claim 7.7.** *For any $i \leq t+1$, let $Z_i$ be the number of indices $j$ such that $(*)$ holds. Then $Z_i \geq \Omega(\log n)$ with high probability.*

*Proof of Claim:* $Z_i$ is the sum of $c \log n$ independent 0-1 random variables, each with mean $\geq \frac{1}{10}$. The expected value is $\mu \geq \frac{c \log n}{10}$. By the Chernoff bound, for a sufficiently large constant $c$, the probability that $Z_i < \frac{c \log n}{20}$ is exponentially small ($o(1/n^3)$). Applying a union bound over all pairs $x, y$ and all $i$, this bound holds uniformly with high probability. Let $K = \frac{c}{20}$, so $Z_i \geq K \log n$.

Using the claim, we can now telescope the sum for the lower bound:

$$\begin{aligned}
d_{\ell_1}(\phi(x), \phi(y)) &= \sum_{i=1}^{\log n} \sum_{j=1}^{c \log n} |d(x, S_{i,j}) - d(y, S_{i,j})| \\
&\geq \sum_{i=1}^{t+1} \sum_{j:(*) \text{ holds}} (r_i - r_{i-1}) \\
&\geq \sum_{i=1}^{t+1} (K \log n)(r_i - r_{i-1}) \\
&= K \log n \sum_{i=1}^{t+1} (r_i - r_{i-1}) \\
&= K \log n (r_{t+1} - r_0) \\
&= K \log n \frac{d(x, y)}{3} \\
&= \Omega(\log n) d(x, y).
\end{aligned}$$

This completes the proof of the lower bound, showing the overall distortion is bounded by $O(\log n)$. $\square$

# 8 March 10: Metric Embedding Day 2

**Definition 8.1** (Tree Metric)**.** *Let $(V, d)$ be a metric space. A tree metric is defined by a tree $T = (V_T, E_T)$ with non-negative edge weights, such that $V \subseteq V_T$. The tree distance $d_T(x, y)$ between any two nodes $x, y \in V$ is the sum of the edge weights along the unique simple path connecting them in $T$.*
*We say $T$ embeds $(V, d)$ with distortion $\alpha \geq 1$ if for all $x, y \in V$:*

$$d(x, y) \leq d_T(x, y) \leq \alpha d(x, y).$$

**Theorem 8.2.** *Let $(X, d)$ be $n$ point space. Suppose $\forall x \neq y \in X$, $d(x, y) \in [1, n^2]$. We will embed $X$ into a tree metric $(T, d_T)$ with*
1. *depth of tree is $O(\log n)$.*
2. *For any root-leaf path, edges weights are decreasing powers of 2.*
3. *Points in $X$ are mapped to leaves of $T$*

*There exist randomized embedding $\phi : X \to (T, d_T)$ such that $\forall x, y \in X$, $d_T(\phi(x), \phi(y)) \geq d(x, y)$ and $\mathbb{E}[d_T(\phi(x), \phi(y))] \leq O(\log n) d(x, y)$*

*Proof.* The algorithm is as follows:

1. Pick random permutation of elements of $X$ say $\pi = \pi_1, ..., \pi_n$
2. Pick uniformly random $\beta \in [1, 2)$ and define $\beta_i := 2^{i-1}\beta$ and for each $x \in X$ and $i = 0, ..., 2 \log n$ and define $center_i(x) :=$ first $\pi_j$ to satisfy $d(x, \pi_j) \leq \beta_i$
3. Construct as follows: Each node represents sets of elements with root at level $2 \log n + 1 =$ all of $X$. For each node $w$ at level $i + 1$, if node $> 1$ element, group $w$'s elements $z$ by $center_i(z)$. Edges from level $i + 1$ to level $i$ has weight $2^i \beta$

Analysis:
We claim: For $x, y \in X$, $d_T(\phi(x), \phi(y)) \geq d(x, y)$. To prove the claim, observe at level $i$, if $x, y$ in same node then $d(x, y) \leq 2\beta_i \leq 2^{i+1}$. Let $i =$ highest level path goes through between $x$ and $y$. $d_T(\phi(x), \phi(y)) \geq 2 \cdot 2^i \geq d(x, y)$.
Property: $\mathbb{E}[d_T(\phi(x), \phi(y))] \leq O(\log n) d(x, y)$
To prove this property, $x := \phi(x), y := \phi(y)$,we can trivially say $d_T(x, y) \leq O(\text{largest edge weight along path})$. This is upper bounded by $\sum_{i=0}^{2 \log n} 2^{i+1} \mathbb{I}[center_i(x) \neq center_i(y)]$. Taking the expectation we get

$$\begin{aligned}
\mathbb{E}[d_T(x, y)] &\leq \sum_{i=0}^{2 \log n} \int_{r=2^{i-1}}^{2^i} \Pr[B_i = r] \Pr[center_i(x) \neq center_i(y) | B_i = r] \cdot 2^{i-1} dr \\
&= \sum_{i=0}^{2 \log n} \int_{r=2^{i-1}}^{2^i} \frac{1}{2^{i-1}} \Pr[center_i(x) \neq center_i(y) | B_i = r] \cdot 2^{i-1} dr \\
&= \sum_{i=0}^{2 \log n} \Pr[center_i(x) \neq center_i(y) | B_i = r] dr \qquad (*)
\end{aligned}$$

Say that $w \in X$ 'cuts' $(x, y)$ at level $i$ if we look through $\pi_1, ...,$ the first of $\{center_i(x), center_i(y)\}$ to appear is $w$ implies $w$ cuts $(x, y)$ and the elements are non-equal.
Let $w_1, w_2, ..., w_n$ be elements of $X$ sorted by distance from $w_i$ to $\{x, y\}$.

$$* \leq \sum_{j=1}^{n} \sum_{i=0}^{2 \log n} \int_{r=2^{i-1}}^{2^i} \Pr[w_j \text{ cuts } (x, y) \text{ at level } i | B_i = r] dr.$$

For $w_j$ to cut $(x, y)$ suppose wlog $d(v_j, x) \leq d(v_j, y)$ we need

1. $d(w_j, x) \leq \beta_i \leq d(w_j, y)$
2. $w_j$ has to appear before $w_1, ..., w_{j-1}$ and this occurs with probability $\frac{1}{j}$

So we can bound the probability as

$$\leq \sum_{j=1}^{n} \int_{d(w_j,x)}^{d(w_j,y)} \frac{1}{j} dz = \sum_{j=1}^{n} \frac{1}{j} \left(d(w_j,y) - d(w_j,x)\right) \leq d(x,y) O(\log n).$$

□

# 9 March 16

**Theorem 9.1.** *There exist history independent algorithm with expected cost* $\tilde{O}\left(\log^{1.5} n\right)$.

**Theorem 9.2.** *There exist history independent algorithm such that if instead is to a uniform random rank, then expected insertion cost is* $\tilde{O}(\log^{1.5} n)$.

**Remark 9.3.** *One potential approach that is incorrect is instead of candidates being median* $\theta(\frac{t}{\log n})$ *elements in subproblem we choose it to be median* $\frac{t}{100\sqrt{\log n \log\log n}}$ *elements in subproblem.*
*The good news is that*

$$\Pr[\textit{insertion into subprob changes pivot}] = O\left(\frac{1}{|Cand|}\right).$$

*This implies the expected cost of insertion is* $\tilde{O}\left(\log^{1.5} n\right)$.
*However the algorithm is not an algorithm as density could be as large as*

$$\frac{1}{2} + \theta(\log n) \cdot \theta\left(\frac{\tilde{1}}{\sqrt{\log n}}\right) >> 1.$$

*Proof.* The fix is that if density(subproblem)$\notin [\frac{1}{4}, \frac{3}{4}]$ revert back to using the $O(\log^2 n)$ solution on this subproblem and all of its descendents. Call this subproblem and its descendents 'corrupt nodes'.
Note: This needs all densities between say $\frac{1}{8}$ and $\frac{7}{8}$. □

**Lemma 9.4.** *Fraction of the leaves that are corrupt is* $\leq \frac{1}{\log^1 0n}$

*Proof.* Let $A_1, A_2, ...$ be the number of paths to the leaf. Let $\triangle_i = density(A_{i+1}) - density(A_i)$.

$$density(A_j) = \frac{1}{2} + \triangle_1 + \cdots + \triangle_{j-1}.$$

We want to show that

$$\left|\sum_{i=1}^{j-1} \triangle_i\right| < \frac{1}{4}.$$

We have that $|\triangle_i| \leq \frac{1}{100\sqrt{\log n \log\log n}}$ and $\mathbb{E}[\triangle_i | \triangle_1, ..., \triangle_{i-1}] = 0$.
Suppose $\triangle_1, \triangle_2, ..., \triangle_\ell \in [-s, s]$ and $\mathbb{E}[\sum_{i=1}^{\ell} \triangle_i] = 0$ then

$$\Pr[|\sum_{i=1}^{\ell} \triangle_i| > K\sqrt{\ell} \cdot S] \leq 2e^{-\frac{K^2}{4}}.$$

For us. $\ell \leq \log n$, $S = \frac{1}{100\sqrt{\log n \log\log n}}$ and set $K = 10\sqrt{\log\log n}$ then

$$2e^{-\frac{K^2}{4}} = 2e^{-2S\log\log n} \approx \frac{1}{(\log n)^{25}} \text{ and } K\sqrt{\ell}S = \frac{1}{10}.$$

So

$$\Pr\left[\left|\sum_{i=1}^{j} \triangle_i\right| \geq \frac{1}{10}\right] \leq \frac{1}{(\log n)^{25}}.$$

So the $\Pr[i\text{-th node } A_i \text{ makes us corrupt}] \leq \frac{1}{(\log n)^{25}}$. By union bound,

$$\Pr[\text{corrupt}] \leq \log n \frac{1}{(\log n)^{25}} \leq \frac{1}{(\log n)^{10}}.$$

□

**Theorem 9.5.** *Consider insertion to random rank with fraction of leaves corrupt being* $\frac{1}{\log^{10} n}$. *Additionally, levels have densities in* $[\frac{1}{8}, \frac{7}{8}]$ *implies*

$$\Pr[\textit{insertion is to corrupt leaf}] = O\left(\frac{1}{\log^1 0n}\right).$$

*If insertion does not hit corrupt node:* $\mathbb{E}[cost] = \tilde{O}(\log^{1.5} n)$.
*If insertion hits correct nodes:* $\mathbb{E}[cost] = O(\log^2 n)$ *and happens with probability* $\frac{1}{\log^1 0n}$.

*Proof.* Consider arbitrary insertion, we can use a modified algorithm

1. Use array of size $4n$
2. Select random $q \in [n]$
3. Add dummy elements $<$ all real elements
4. Add $n - q$ dummy elements $>$ all real elements

The rank including dummy elements is $q + r \in [r, n + r]$.

$$\mathbb{E}[cost] = \sum_{i=r}^{n+r} \mathbb{E}[\text{cost of ins rank } i] O\left(\frac{1}{n}\right) \leq \sum_{i=1}^{2n} O\left(\frac{1}{n}\right) \mathbb{E}[\text{cost of ins at rank } i] = \tilde{O}(\log^{1.5} n).$$

□

# 10 March 18 — Optional Stopping Theorem

**Definition 10.1.** *Consider sequence of random variables $X_0, X_1, ..., X_S$ (not random)*

1. *$X_0$ is amount you start with*
2. *$X_i$ = amount you have after $i$ steps*
3. *$S$ is the stopping time and is a random variable determined by $X_0, ..., X_s$ and is finite with probability 1.*

*We require for any $i > 0$*

$$\mathbb{E}[X_i | X_0, X_1, ..., X_{i-1}, S > i-1] = X_{i-1}.$$

**Theorem 10.2.** *Optional Stopping Theorem: If $\mathbb{E}[S]$ finite and $\exists C$ such that for all $i, \mathbb{E}[|X_i - X_{i-1}||S > i-1] \leq C$. Then $\mathbb{E}[X_S] = X_0$*

*Proof.* Define $\Delta_i = X_i - X_{i-1}$, wewant to show that $\mathbb{E}[\sum \delta_i] = 0$. Define $\Delta_i^+ = \begin{cases} \Delta_i & \text{if } \Delta_i \geq 0 \\ 0 & \text{if } \Delta_i < 0 \end{cases}$ and $\Delta_i^- = \begin{cases} \Delta_i & \text{if } \Delta_i < 0 \\ 0 & \text{if } \Delta_i \geq 0 \end{cases}$ and note that $\Delta_i = \Delta_i^+ - \Delta_i^-$.

$$\mathbb{E}\left[\sum \Delta_i^+\right] = \sum \mathbb{E}\left[\Delta_i^+\right] \leq C \sum \Pr[S \geq i] = C\mathbb{E}[S].$$

Likewise, $\mathbb{E}\left[\sum \Delta_i^-\right]$ is finite.

$$\mathbb{E}[\sum \Delta_i] = \mathbb{E}\left[\sum \Delta_i^+ - \sum \Delta_i^-\right] = \mathbb{E}\left[\sum \Delta_i^+\right] - \mathbb{E}\left[\sum \Delta_i^-\right] = 0.$$

□

**Example 10.3.** *Consider the game*

1. *Start at $n$*
2. *On each step, go left with prob $\frac{2}{3}$ and right with prob $\frac{1}{3}$*
3. *Stop when you get to 0*

*Let $S = \#$ steps until you get to 0. What is $\mathbb{E}[S]$.*
*Let $P_i$ = position at step $i$ and define $X_i = P_i + \frac{1}{3}i$ then*

$$\mathbb{E}[X_i | ...] = X_{i-1} + 1 \cdot \frac{1}{3} + (-1) \cdot \frac{2}{3} + \frac{1}{3} = X_{i-1}.$$

*We have $n = X_0 = \mathbb{E}[X_S] = \mathbb{E}\left[P_S + \frac{S}{3}\right] = \frac{1}{3}\mathbb{E}[S]$ implying $\mathbb{E}[S] = 3n$.*

# 11 March 23

**Example 11.1.** *Start at* 1 *and perform a random walk where* $+1$ *with probability* $\frac{2}{3}$ *and* $-1$ *with probability* $\frac{1}{3}$.
*What is* $\Pr[$*get to* $n$ *before* $0]$*?*

*Proof.* Define $X_t =$ position after $t$ steps and $Y_t = \frac{1}{2^{X_t}}$.
Observe that $\mathbb{E}[Y_{t+1}|Y_t] = Y_t$.
Let $Y_S$ be the point where we hit 0 or $n$. By optional stopping theorem, $\mathbb{E}[Y_S] = Y_0 = \frac{1}{2}$ and

$$\mathbb{E}[Y_S] = p \cdot \frac{1}{2^n} + (1-p) \cdot 1.$$

So $p = \frac{1}{2(1-\frac{1}{2^n})}$ □

# 12 March 25

## 12.1 Linear probing hash table — setup

We store a set of at most $n$ keys drawn from a universe $\mathcal{U}$ in a table $T$ of $m$ slots indexed by $\{0, 1, \ldots, m-1\}$. Each slot holds at most one key (or is empty). We use a hash function $h\colon \mathcal{U} \to \{0, \ldots, m-1\}$; in the randomized model, $h$ is drawn from a suitable family (e.g. a universal or strongly universal family), or we analyze a fully random function $h$.

**Insertion.** To insert key $k$, examine slots

$$h(k),\ h(k)+1,\ h(k)+2,\ \ldots \pmod m$$

in order until finding an empty slot, and place $k$ there.

**Query.** To search for $k$, probe the same sequence starting at $h(k)$ until either finding $k$ or hitting an empty slot (then $k$ is absent).

**Theorem 12.1.** *Knuth 62': Suppose we fill hash table to $(1-\frac{1}{x})$ load then the expected insertion time is $O\left(x^2\right)$.*

**Lemma 12.2.** *Suppose we filled Hash Table to $\left(1-\frac{1}{x}\right)$ full. We query random element out of those present, the expected query time is $O(x)$.*

**Recall 12.3.** *Poisson Bus Arrivals:*
1. *In any interval of length $\ell$, $\mathbb{E}[\#arrivals] = \ell$.*
2. *For any set of disjoint interval arrivals in these intervals are mutually independent.*

**Lemma 12.4.** *If $X_1, \ldots. X_n \sim$ Poisson$\left(\frac{\mu}{n}\right)$ independent then $X_1 + \ldots + X_n \sim$ Poisson$(\mu)$.*

**Lemma 12.5.** *An equivalent process is to first sample $X \sim$ Poisson$(\mu)$ and then place $X$ bus arrivals randomly in $[0, \mu]$*

**Lemma 12.6.** *If I throw Poisson$(\mu)$ balls into $n$ bins. Let $X_i = \#$ balls in bin $i$. Then $X_1, \ldots. X_n$ are mutually independent and $X_i \sim$ Poisson$\left(\frac{\mu}{n}\right)$.*

*Proof.* By previous lemma, we can think of $X_i = \#$ arrival in the $i$-th subinterval. Then each subinterval is poisson distribution. □

In our setting, let $X_i = \#$ elements that hash $h(v) = i$, but rather than inserting $\left(1-\frac{1}{x}\right)n$ elements, insert Poisson $((1-1/x)n + 100\log n\sqrt{n}$ elements.
Note: This poisson is $> (1-1/x)n$ with high probability in $n$.
For the rest of the proof, assume we do Poisson$((1-1/x)n)$ insertions.

**Lemma 12.7.** *If reorder elements in a run, this does not change the expected query time.*

*Proof.* If swap 2 elements implies the average stays the same. □

Suppose we order elements by giving priority to elements with bigger hashes.
The time to query elements is at most $\arg\min_S((\#\text{elements besides } u \text{ with hash in } [h(u), h(u)+S]) < S+1)$

# 13 April 1: Succinct Data Structures

**Theorem 13.1.** *We can build a HT that stores up to $n$ keys ($w$-bit integers and we are in word-RAM model with words of size $w$)*

1. *$O(1)$ exp time*
2. *AND, space usage $\leq \left(1 + \frac{1}{\log n}\right) nw + O(n \log\log n)$ bits $= (1 + o(1))nw$ bits.*

**Theorem 13.2.** *This can be done in $\left(1 + \frac{1}{\log n}\right) nw + O(n \log\log n) - n \log n$ bits.*

*Proof.* To save $\approx n \log n$ bits, we use quotienting by pretending each element looks like $a + b$ where $|a| = \log m$ bits and $|b| = w - \log m$ bits for $m = \frac{n}{\text{bin size}}$. $a$ is randomized so we can just use this as our hash table. So go to bucket $a$ and store only $b$. However the issue is we don't know how to randomize $a$. The solution is to think of each elements as pairs $(a, b)$. Let *hbeah* $\square$